



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



TFG del Grado en Ingeniería
Informática

Evolución y Optimización de
la Digitalización del Boletín
de Turismo de la Provincia de
Burgos



Presentado por Nicolás Pérez Ibáñez
en Universidad de Burgos — 7 de julio de 2026

Tutor: Bruno Baruque Zanón

Cotutor: Julio César Puche Regaliza



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



D. Bruno Baruque Zanón, profesor del departamento de Digitalización, área de Ciencia de la Computación e Inteligencia Artificial y D. Julio César Puche Regaliza, profesor del departamento de Economía Aplicada, área de Métodos Cuantitativos para la Economía y la Empresa.

Exponen:

Que el alumno D. Nicolás Pérez Ibáñez, con DNI 71311265H, ha realizado el Trabajo final de Grado en Ingeniería Informática titulado Evolución y Optimización de la Digitalización del Boletín de Turismo de la Provincia de Burgos.

Y que dicho trabajo ha sido realizado por el alumno bajo la dirección del que suscribe, en virtud de lo cual se autoriza su presentación y defensa.

En Burgos, 7 de julio de 2026

Vº. Bº. del Tutor:

Vº. Bº. del co-tutor:

D. Bruno Baruque Zanón

D. Julio César Puche Regaliza

Resumen

Este proyecto presenta la evolución y optimización de la digitalización del Boletín de Turismo de la Provincia de Burgos mediante el diseño de una infraestructura de datos completamente automatizada y resiliente. El sistema sustituye los procesos manuales por un flujo continuo en Python que extrae puntos de interés y reseñas, limpia y traduce los textos automáticamente, y calcula el Índice de Reputación Online Turística (TORI). La información se estructura en Google BigQuery y se muestra visualmente en Looker Studio a través de un cuadro de mando interactivo. Con todo ello, el proyecto logra consolidar una herramienta analítica centralizada y dinámica que refleja la percepción real de los turistas, facilitando una toma de decisiones estratégicas basadas en datos que ayude a potenciar y dinamizar el sector turístico de la región.

Descriptores

pipeline, ETL, puntos de interés, reseñas, municipios, categorías, web scraping, Apify, Google Maps, sentimiento, procesamiento del lenguaje natural, Python, BigQuery, modelo relacional, SQL, Looker Studio, visualización de datos, boletín de turismo, TORI, automatización, nube, GitHub Actions, workflow, Docker, contenerización, Git, turismo, Burgos.

Abstract

This project presents the evolution and optimization of the digitalization of the Burgos Province Tourism Bulletin through the design of a fully automated and resilient data infrastructure. The system replaces manual workflows with a seamless Python pipeline that extracts points of interest and user reviews, automatically refines and translates textual data, and derives the Tourism Online Reputation Index (TORI). This information is structured within Google BigQuery and visualized via an interactive Looker Studio dashboard. Ultimately, the project delivers a centralized, dynamic analytical tool that mirrors genuine tourist perception, facilitating data-driven strategic decision-making to help boost and revitalize the regional tourism sector.

Keywords

pipeline, ETL, points of interest, reviews, municipalities, categories, web scraping, Apify, Google Maps, sentiment, natural language processing, Python, BigQuery, relational model, SQL, Looker Studio, data visualization, tourism bulletin, TORI, automation, cloud, GitHub Actions, workflow, Docker, containerization, Git, tourism, Burgos.

Índice general

Índice general	iii
Índice de figuras	v
1. Introducción	1
1.1. Contexto del proyecto	1
1.2. Resumen del contenido del proyecto	1
1.3. Estructura de la documentación	2
1.4. Materiales entregados	4
2. Objetivos del proyecto	7
2.1. Objetivos funcionales	7
2.2. Objetivos técnicos	8
3. Conceptos teóricos	9
3.1. Algoritmo ETL	9
3.2. Interfaces de Programación de Aplicaciones (APIs)	11
3.3. Modelos de almacenamiento: Bases de datos relacionales y analíticas	13
3.4. Procesamiento del lenguaje natural y análisis de sentimiento	14
3.5. Índice TORI	15
4. Técnicas y herramientas	17
4.1. Metodología de trabajo	17
4.2. Sistema de control de versiones (Git)	18
4.3. Plataforma central de desarrollo y automatización (GitHub)	18
4.4. Entorno de desarrollo local (Visual Studio Code)	20

4.5. Herramienta de extracción (Apify)	21
4.6. Infraestructura de almacenamiento (Google BigQuery)	22
4.7. Contenedorización (Docker)	23
4.8. Lenguajes de programación	24
4.9. Bibliotecas de procesamiento de datos y conectores	24
4.10. Herramienta de cuadro de mando y visualización (Looker Studio)	25
4.11. Herramientas de diseño gráfico y diagramas (Draw.io)	26
4.12. Herramientas de asistencia (Gemini)	26
5. Aspectos relevantes del desarrollo del proyecto	29
5.1. Estructura y división del código en archivos independientes	29
5.2. Protección contra fallos y sistema de checkpoints)	30
5.3. Desacoplamiento y configuración externa del sistema	32
5.4. Optimización de costes de la API según los habitantes del municipio	32
5.5. Descubrimiento y guardado dinámico de nuevas categorías	33
5.6. Logging	34
6 Trabajos Relacionados	35
6.1. Comparativa con el proyecto anterior	35
6.2. Marcos de referencia nacionales	36
7. Conclusiones y Líneas de trabajo futuras	39
7.1. Conclusiones del proyecto	39
7.2. Líneas de trabajo futuras	40
Bibliografía	43

Índice de figuras

3.1. Representación de las tres fases del proceso ETL (Extracción, Transformación y Carga).	10
3.2. Flujo de comunicación basado en arquitectura API REST utilizando métodos HTTP y formato de datos JSON.	12
4.1. Logotipo del sistema de control de versiones Git.	18
4.2. Logotipo de la plataforma GitHub.	18
4.3. Logotipo de la plataforma de gestión Zenhub.	19
4.4. Logotipo del motor de automatización GitHub Actions.	19
4.5. Logotipo del entorno de desarrollo Visual Studio Code.	21
4.6. Logotipo de la plataforma de extracción web Apify.	22
4.7. Logotipo del almacén de datos Google BigQuery.	23
4.8. Logotipo de la plataforma de contenedores Docker.	23
4.9. Logotipo de la herramienta de visualización Looker Studio.	26
4.10. Logotipo de la herramienta de diagramas Draw.io.	26
4.11. Logotipo del asistente técnico Gemini.	27

1. Introducción

1.1. Contexto del proyecto

La provincia de Burgos cuenta con una gran extensión territorial y una marcada dispersión geográfica, caracterizada por la existencia de una gran cantidad de pequeños municipios rurales junto a unos pocos núcleos urbanos concentrados. En este escenario, el turismo sostenible y de interior se ha convertido en un motor clave para la revitalización económica y la creación de empleo, actuando como una herramienta directa frente al reto de la despoblación rural. Sin embargo, para que las administraciones y los gestores locales puedan tomar decisiones estratégicas eficaces, es imprescindible conocer de primera mano la situación y la reputación online de sus recursos turísticos.

Llevar a cabo una recopilación de datos de forma manual, revisando uno a uno las opiniones, valoraciones y el estado de los puntos de interés (POIs) repartidos por toda la provincia, resulta una tarea totalmente inviable en términos de tiempo y costes económicos. Es aquí donde se justifica el desarrollo de este proyecto, que propone la creación de un sistema automatizado capaz de monitorizar de forma masiva, continua e incremental la huella digital del sector turístico burgalés a través de plataformas como Google Maps, transformando los datos cualitativos dispersos en información estructurada y de alto valor analítico.

1.2. Resumen del contenido del proyecto

El núcleo de este Trabajo Fin de Grado consiste en el diseño y desarrollo de un pipeline (línea de procesamiento) ETL (Extracción, Transformación y Carga) completamente automatizado y resiliente. El sistema inicia su

flujo conectándose con la API externa de Apify, encargada de realizar el la extracción de datos masiva y automatizada en la web de las fichas comerciales o lugares de interés y sus respectivas reseñas de Google Maps.

Una vez extraídos los datos en bruto, el programa ejecuta de forma local una serie de transformaciones basadas en el procesamiento del lenguaje natural (NLP). Entre estas tareas se incluye un filtro de calidad de texto, la traducción automática de reseñas internacionales, la estimación del género del autor por su nombre y un análisis predictivo del sentimiento de los comentarios. Finalmente, los datos enriquecidos se cargan en bloque de forma eficiente en un almacén de datos analítico en Google BigQuery. Esta base de datos se conecta de manera directa con un cuadro de mando interactivo en Looker Studio, permitiendo actualizar los gráficos de reputación del Boletín de Turismo en tiempo real y sin intervención humana.

1.3. Estructura de la documentación

El cuerpo documental de este trabajo se compone de la memoria principal, organizada en siete capítulos, y un conjunto de anexos técnicos que profundizan en los detalles de la especificación e implementación:

Los capítulos que forman la memoria técnica principal se estructuran de la siguiente manera: Los capítulos que forman la memoria técnica principal se estructuran de la siguiente manera:

- **Capítulo 1 (Introducción):** Define el marco inicial del proyecto, justificando la motivación del software frente a la situación turística regional y enumerando el contenido global de la documentación.
- **Capítulo 2 (Objetivos del proyecto):** Detalla la meta general del sistema y los hitos técnicos específicos que se pretenden alcanzar en cuanto a optimización, almacenamiento y automatización.
- **Capítulo 3 (Conceptos teóricos):** Expone los fundamentos del sistema, detallando el proceso ETL, el funcionamiento de las APIs y técnicas de *web scraping*, las diferencias entre bases de datos relacionales y analíticas, los modelos de procesamiento del lenguaje natural para el análisis de sentimiento y el cálculo matemático del índice TORI.
- **Capítulo 4 (Técnicas y herramientas):** Presenta la metodología ágil basada en *Sprints* y Kanban, y justifica la elección de toda

la infraestructura tecnológica utilizada: desde el control de versiones y automatización en la nube (Git y GitHub Actions) hasta las herramientas de extracción (Apify), almacenamiento (BigQuery), contenedorización (Docker) y visualización (Looker Studio), detallando además los lenguajes y librerías empleados.

- **Capítulo 5 (Aspectos relevantes del desarrollo del proyecto):** Explica la implementación práctica del software, detallando la división modular del código en archivos independientes, la tolerancia a fallos mediante el sistema de *checkpoints* por lotes, el desacoplamiento lógico mediante el archivo `config.json`, la regla adaptativa de precisión según la población, el descubrimiento dinámico de categorías dentro de la taxonomía y la integración del sistema estructurado de *logging*.
- **Capítulo 6 (Trabajos relacionados):** Compara este proyecto con el antecedente directo del Trabajo de Fin de Grado presentado el año anterior, analizando las mejoras estratégicas en almacenamiento, automatización y visualización, y expone la alineación del pipeline con el marco nacional de Destinos Turísticos Inteligentes (DTI) gestionado por SEGITTUR.
- **Capítulo 7 (Conclusiones y líneas de trabajo futuras):** Resume la experiencia práctica del proyecto, evalúa el cumplimiento de los objetivos desde una perspectiva personal y plantea mejoras para la evolución del sistema.

Por otra parte, se adjuntan los siguientes apéndices con la documentación técnica y de soporte en el documento de los anexos:

- **Apéndice A (Plan de proyecto):** Contiene la planificación temporal detallada del proyecto organizada por sprints de desarrollo y el estudio de viabilidad económica correspondiente.
- **Apéndice B (Especificación de requisitos):** Define los objetivos generales del sistema junto con el catálogo de requisitos funcionales y no funcionales, incluyendo el diagrama de casos de uso y sus tablas descriptivas individuales.
- **Apéndice C (Especificación de diseño):** Detalla el plano técnico del sistema mediante tres enfoques: el diseño de datos con su diagrama entidad-relación, el diseño arquitectónico y el diseño procedimental mediante un diagrama de secuencia que representa el funcionamiento interno del código en tiempo de ejecución.

- **Apéndice D (Manual del programador):** Recoge toda la documentación técnica para desarrolladores, detallando el entorno de desarrollo, la estructura de directorios, el funcionamiento de las herramientas, el proceso de instalación y ejecución, y el plan de pruebas del sistema.
- **Apéndice E (Manual de usuario):** Sirve como guía práctica orientada al usuario final para explicar el funcionamiento y la interacción con el cuadro de mando de Looker Studio.
- **Apéndice F (Objetivos de Desarrollo Sostenible):** Analiza el impacto del proyecto y su relación directa con los Objetivos de Desarrollo Sostenible (ODS) de las Naciones Unidas.

1.4. Materiales entregados

Junto con la documentación de la memoria y los anexos, se aporta un repositorio digital estructurado que contiene todas las herramientas y recursos necesarios para la reconstrucción, configuración y ejecución del sistema desarrollado. Los materiales entregados se distribuyen de la siguiente forma:

- **Código fuente de la aplicación (src/):** Contiene los scripts desarrollados en Python 3.13 encargados del control global del pipeline (`main_scraper_pipeline.py`), la extracción de datos (`apify_extractor.py`), la analítica y limpieza de las reseñas extraídas (`review_transformer.py`) y la carga en la nube de la información (`bigquery_loader.py`), junto con los scripts que ejecutan el archivo de configuración y el logger.
- **Módulo de base de datos (database/):** Incluye el archivo (`schema.sql`) con las sentencias de creación de las tablas y vistas lógicas en Google BigQuery, acompañado de los ficheros CSV de carga inicial para rellenar los datos maestros de municipios, categorías y códigos postales.
- **Estructura de configuración (config/):** Archivo JSON centralizado (`config.json`) que permite al usuario modificar de forma inmediata los filtros de ejecución, los márgenes de días para las actualizaciones, los identificadores de las tablas, etc.
- **Infraestructura y despliegue:** Archivos preconfigurados para la contenerización del sistema en local mediante Docker (`Dockerfile` y

`docker-compose.yml`) y para la automatización de la ejecución programada en la nube mediante GitHub Actions (`run_scraper.yml`).

- **Cuadro de mando:** Enlace de acceso y configuración del informe interactivo diseñado en Looker Studio, preparado para leer de forma dinámica las vistas semánticas del almacén de datos de BigQuery.

2. Objetivos del proyecto

Este capítulo explica cuáles son los objetivos que se persiguen con la realización del proyecto. Para estructurar correctamente el diseño del sistema desde la perspectiva de la ingeniería de software, estas metas se dividen en dos grandes bloques: aquellos objetivos marcados de forma directa por los requisitos funcionales del software que se va a construir, y aquellos objetivos de carácter puramente técnico y metodológico que plantea la puesta en práctica e implementación del pipeline en un entorno de producción real.

2.1. Objetivos funcionales

Los objetivos funcionales definen las tareas analíticas y de procesamiento de datos que la aplicación debe resolver para cumplir las necesidades del Observatorio de Turismo:

- **Extracción automatizada de información turística:** El sistema debe ser capaz de recopilar de forma automática los puntos de interés de la provincia de Burgos y todas sus reseñas asociadas directamente desde Google Maps, obteniendo la información detallada de cada uno.
- **Clasificación taxonómica por niveles:** Desarrollar una lógica interna para organizar y clasificar las categorías de Google Maps en varios niveles establecidos, asignando correctamente cada categoría a su correspondiente punto de interés.
- **Cálculo del TORI:** Implementar en el sistema las fórmulas necesarias para calcular el índice de reputación online TORI de los establecimientos, permitiendo valorar el nivel de satisfacción real de los visitantes.

- **Panel de visualización interactivo:** Diseñar un cuadro de mando centralizado que unifique los datos extraídos para que los analistas puedan consultar, filtrar y estudiar la información provincial de forma visual.

2.2. Objetivos técnicos

Los objetivos técnicos detallan las herramientas utilizadas, la integración y las restricciones tecnológicas que condicionan el desarrollo:

- **Restricción de la plataforma de visualización:** Debido a las limitaciones de infraestructura del servidor del Observatorio de Turismo, se establece el uso obligatorio de Looker Studio como herramienta de visualización para asegurar su futura integración web.
- **Automatización completa en la nube:** Diseñar la arquitectura para que el sistema funcione de manera autónoma y periódica sin depender de la ejecución manual desde un ordenador personal, logrando que el sistema se active, actualice la base de datos y refresque el dashboard por sí solo.
- **Almacenamiento analítico gestionado:** Utilizar un almacén de datos en la nube (BigQuery) en lugar de una base de datos relacional local tradicional. Esta elección técnica busca garantizar que la información esté siempre accesible y conectada de forma directa con la capa de visualización.
- **Portabilidad mediante contenedores:** Empaquetar todo el entorno de desarrollo y las librerías lógicas del proyecto utilizando contenedores de Docker, asegurando que el software sea reproducible y funcione exactamente igual independientemente del ordenador o servidor donde se despliegue.

3. Conceptos teóricos

En este capítulo se describen los conceptos técnicos y los fundamentos computacionales necesarios para comprender el funcionamiento de los sistemas modernos de información. En lugar de analizar herramientas particulares, este apartado explica de forma teórica los flujos de datos, los mecanismos de comunicación entre programas, los modelos de almacenamiento y las técnicas de análisis de texto que se utilizan en la ingeniería de software actual.

3.1. Algoritmo ETL

En el ámbito de la ingeniería de datos, es habitual necesitar mover información desde diferentes lugares de origen hacia un almacén centralizado para poder analizarla [7]. El proceso estándar para realizar esta tarea se conoce como ETL, que son las siglas de Extracción, Transformación y Carga. Este proceso funciona como una línea de producción dividida en tres etapas independientes:

Extracción

La fase de extracción consiste en obtener los datos en bruto desde las fuentes originales, que pueden ser archivos de texto, páginas web o bases de datos externas. El objetivo principal de esta etapa es leer la información de forma segura sin alterar el origen. Los sistemas automáticos de extracción deben estar preparados para gestionar problemas comunes como cortes en la conexión a internet, lentitud en las respuestas o la recepción de datos incompletos. En esta fase, el sistema simplemente se encarga de descargar

Proceso ETL

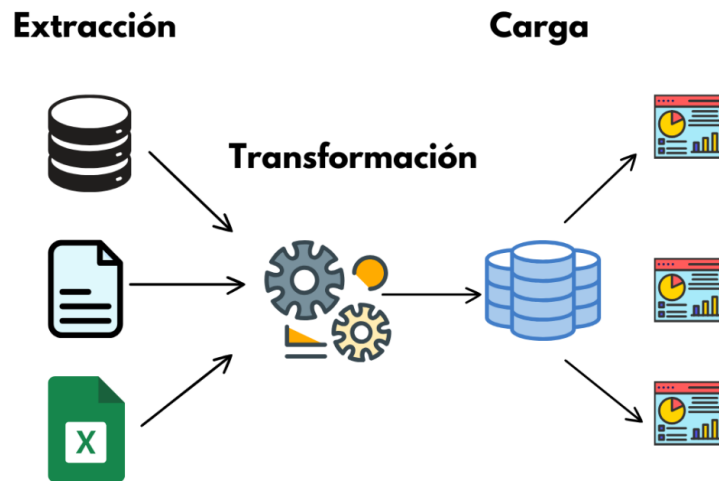


Figura 3.1: Representación de las tres fases del proceso ETL (Extracción, Transformación y Carga).

la información y dejarla temporalmente en una zona de memoria para su posterior procesamiento.

Transformación

La transformación es la etapa más compleja y donde se aplica la lógica del sistema. Los datos descargados en bruto rara vez están listos para ser utilizados directamente, ya que suelen contener errores o formatos diferentes. En esta fase se realizan tres tareas fundamentales:

- **Limpieza de datos:** Se eliminan los registros duplicados, se corrigen los textos dañados y se decide qué hacer con los valores que están vacíos.
- **Homogeneización:** Se unifican los formatos. Por ejemplo, se configuran todas las fechas para que sigan el mismo orden (Año-Mes-Día) o se transforman los textos a minúsculas para facilitar las búsquedas.

3.2. INTERFACES DE PROGRAMACIÓN DE APLICACIONES (APIs)

- **Enriquecimiento:** Se añade valor a los datos originales. Esto se consigue realizando cálculos matemáticos o aplicando modelos inteligentes para generar información nueva a partir de la ya existente.

Carga

La fase de carga consiste en guardar de forma definitiva los datos ya limpios y transformados en el almacén final. Existen dos formas de hacerlo: la carga completa, que borra lo anterior y escribe todo de nuevo, y la carga incremental. La carga incremental es mucho más eficiente porque consiste en revisar qué datos son nuevos o han cambiado desde la última vez que se ejecutó el proceso, añadiendo únicamente esas filas. Para evitar que los datos se dupliquen, se utilizan instrucciones lógicas que comprueban si el registro ya existe en el almacén. Si ya existe se actualiza, y si no, se inserta como uno nuevo.

3.2. Interfaces de Programación de Aplicaciones (APIs)

Una Interfaz de Programación de Aplicaciones, conocida habitualmente como API, es un conjunto de reglas y funciones que permite que dos programas informáticos se comuniquen entre sí de forma directa. Se puede entender como un puente o un intermediario técnico: un programa le pide datos a la API (petición) y la API le devuelve la información solicitada (respuesta) en un formato organizado que ambos ordenadores entienden, como por ejemplo el formato JSON^[10].

Las APIs sirven para que los desarrolladores puedan aprovechar los datos o las funciones de otras empresas sin necesidad de saber cómo están programadas por dentro. Por ejemplo, permiten pedir la información del tiempo a un servicio meteorológico o consultar mapas de una empresa externa.

A pesar de su utilidad, cuando se diseñan sistemas que necesitan descargar un volumen muy grande de datos históricos, las APIs suelen presentar limitaciones importantes impuestas por los propios proveedores:

- **Límites de peticiones:** Para evitar que sus servidores se colapsen, los proveedores limitan el número de veces que se puede llamar a la API al día o al minuto.

- **Restricciones de cantidad:** Muchas APIs restringen el número de resultados devueltos. Por ejemplo, al pedir opiniones de un lugar, el sistema puede estar limitado a entregar únicamente las 5 más recientes, haciendo imposible consultar el histórico completo.
- **Costes financieros:** El acceso a datos masivos a través de APIs oficiales suele estar sujeto a tarifas de pago por cada petición realizada, lo que puede encarecer los proyectos con presupuestos limitados.

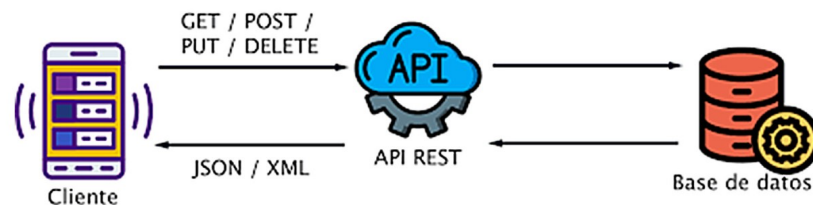


Figura 3.2: Flujo de comunicación basado en arquitectura API REST utilizando métodos HTTP y formato de datos JSON.

Técnicas de extracción web (Web Scraping)

Cuando las APIs oficiales no existen o están demasiado limitadas para el objetivo del proyecto, se utilizan las técnicas de extracción web o *Web Scraping*. Esta metodología consiste en programar un script automático que actúa como si fuera un navegador web utilizado por una persona. El programa entra en la página web, imita acciones humanas (como hacer clic en botones o desplazarse hacia abajo en la pantalla) y lee la información directamente del código de la página para guardarla de forma estructurada [15].

Las páginas web actuales no son estáticas, sino que cargan su contenido poco a poco a medida que interactuamos con ellas. Por ello, los programas de extracción modernos deben ser capaces de interpretar toda la estructura

interna de la página web (el árbol de elementos visuales) y buscar la información utilizando identificadores o etiquetas específicas del diseño. Esto permite saltarse las restricciones de espacio de las APIs convencionales y capturar la información tal y como se muestra en la pantalla.

3.3. Modelos de almacenamiento: Bases de datos relacionales y analíticas

La forma en que se guardan los datos en el disco duro determina la velocidad a la que el sistema podrá consultar la información después. En la ingeniería de software existen dos formas principales de organizar el almacenamiento según el uso que se le vaya a dar al sistema [6]:

Bases de datos tradicionales (Orientadas a filas)

Son las bases de datos relacionales de toda la vida, diseñadas para el trabajo diario de las aplicaciones (como registrar un usuario o guardar una compra). Físicamente, guardan toda la información de una misma fila junta en el disco duro. Esto es ideal para buscar, escribir o modificar la ficha completa de un elemento concreto muy rápido. Sin embargo, si un analista quiere calcular la media de una columna numérica entre millones de registros, el ordenador se ve obligado a leer todas las filas enteras de la base de datos con todos sus textos y campos adicionales, lo que satura la memoria y vuelve el proceso muy lento.

Bases de datos analíticas (Orientadas a columnas)

Son almacenes de datos optimizados para analizar grandes volúmenes de información histórica y crear cuadros de mando. A diferencia de las tradicionales, estas bases de datos rompen las tablas verticalmente y guardan cada columna en un archivo independiente en el disco duro. Si un programa necesita hacer una suma o calcular la media de una variable, el motor del sistema va directamente al archivo de esa columna y lee solo esos números, ignorando por completo el resto de los textos o datos de la tabla. Esto reduce de forma drástica la cantidad de información que procesa el ordenador, permitiendo responder a consultas complejas en pocos segundos y con un consumo mínimo de recursos.

3.4. Procesamiento del lenguaje natural y análisis de sentimiento

El Procesamiento del Lenguaje Natural (NLP) es la rama de la inteligencia artificial que busca que los ordenadores entiendan, interpreten y operen con el lenguaje humano tal y como lo hacemos las personas. Como los ordenadores solo entienden de matemáticas y números, el texto escrito debe pasar por un proceso de traducción numérica antes de poder ser analizado.

El concepto de Tokenización

La tokenización es el paso inicial para que un ordenador pueda procesar cualquier texto. Consiste en trocear una cadena de texto continua en unidades mínimas con significado llamadas *tokens*. En lugar de separar las frases simplemente por palabras completas (lo que daría problemas con las faltas de ortografía o las palabras inventadas), los sistemas modernos rompen el texto en pequeñas sílabas o fragmentos comunes de letras.

Una vez que el texto está troceado, a cada fragmento se le asigna un número identificador único según un diccionario estático. De esta forma, una frase de texto libre se convierte en una lista de números. Tras este paso, el sistema traduce esos números en coordenadas dentro de un espacio matemático, permitiendo que las palabras que tienen significados similares o que suelen ir juntas queden guardadas cerca las unas de las otras.

Modelos basados en contexto (Transformers)

Los sistemas antiguos analizaban las palabras de una frase una a una de forma aislada, lo que hacía que no entendieran el significado real cuando había negaciones (como "no es bueno") o ironías. Los modelos actuales utilizan un mecanismo llamado atención [9]. Este principio permite al sistema analizar todas las palabras de una frase al mismo tiempo y calcular matemáticamente cómo influye cada palabra en el significado de las demás. Así, el ordenador entiende el contexto global de la frase antes de tomar una decisión.

Cálculo del sentimiento

El análisis de sentimiento aprovecha esta comprensión del contexto para clasificar la carga emocional de un texto. El modelo analiza los números de la frase filtrados por el mecanismo de atención y calcula la probabilidad de que el comentario sea positivo, neutro o negativo. Para poder trabajar con

este dato en ingeniería de software, estas probabilidades se transforman en una nota numérica continua llamada puntuación de sentimiento. Esta nota se ajusta obligatoriamente en un rango que va desde -1 (para los textos con un enfado o insatisfacción absoluta) hasta 1 (para los textos con una satisfacción máxima), siendo el valor 0 para los comentarios neutrales [13].

3.5. Índice TORI

En la analítica de datos, las puntuaciones simples que ponen los usuarios (como las notas de 1 a 5 estrellas en Google Maps) no siempre reflejan la realidad de forma justa. Esto ocurre por el sesgo de los extremos: las personas tienden a dejar una nota en internet únicamente cuando están muy enfadadas o muy contentas, por lo que la media de estrellas suele estar desvirtuada.

Para corregir esto, se utilizan los indicadores compuestos, que consisten en una fórmula matemática que junta varias medidas diferentes para sacar una única nota final más equilibrada. El *Tourism Online Reputation Index* (TORI) es un índice diseñado para el sector turístico que calcula la reputación real de un punto de interés combinando dos fuentes de información: la valoración de estrellas física de la plataforma y la media del sentimiento de los textos analizada por el ordenador. Como estas puntuaciones vienen dadas originalmente en escalas y amplitudes diferentes, el modelo requiere un proceso de escalamiento matemático para poder compararlas:

- **Amplitud de las estrellas:** Las valoraciones de Google Maps van del 1 al 5, lo que representa una amplitud de 4 puntos.
- **Amplitud del sentimiento:** El cálculo del análisis de sentimiento puede tener diferentes rangos de negativo a positivo, por ejemplo de -1 a 1 , lo que representa una amplitud de 2 puntos.

Para que ambas partes compartan una escala homogénea, el diseño del sistema realiza un escalado automático basado en dos toques numéricos variables fijados en la configuración del sistema (`max_score_rating` y `max_score_sentiment`). Estos dos números representan el total de puntos máximos que se le quiere dar a cada componente dentro del índice global, lo que permite modificar el peso o la importancia de cada una de las partes según los criterios del analista, asegurando que el mínimo de ambas sea siempre 0 para facilitar su comparación.

Por ejemplo, si a la valoración de estrellas se le asigna un tope de 7 y al sentimiento un tope de 3, la escala final del índice TORI se moverá en un rango de 0 a 10, otorgando un 70 % de importancia a las estrellas y un 30 % al texto de las opiniones. Si se prefiere una escala sobre 100, bastará con indicar una combinación de topes que sumen dicho valor (como 50 y 50), generando un rango de TORI entre 0 y 100.

Para ejecutar este comportamiento de forma dinámica, el sistema calcula en tiempo de ejecución dos constantes de proporcionalidad (k_{rating} y $k_{\text{sentiment}}$):

- La constante k_{rating} se obtiene dividiendo el tope deseado para las estrellas entre 4 (su amplitud original).
- La constante $k_{\text{sentiment}}$ se obtiene dividiendo el tope deseado para el sentimiento entre 2 (su amplitud original).

A partir de estas constantes, el cálculo matemático aplica un desplazamiento lineal a las variables originales para unificar sus puntos de partida en una base cero común:

- A la nota media de estrellas (`poi_total_rating`) se le resta 1 para desplazar su rango de [1, 5] a [0, 4], multiplicándose después por su constante para expandirla al tope configurado.
- A la media aritmética de las puntuaciones de sentimiento (`sentiment_score`) se le suma 1 para eliminar los valores negativos y desplazar su rango de [-1, 1] a [0, 2], multiplicándose igualmente por su constante de expansión.

La fórmula matemática definitiva que unifica este diseño y calcula el valor del índice final se estructura de la siguiente manera:

$$\text{tori_score} = (k_{\text{rating}} \times (\text{poi_total_rating} - 1)) + (k_{\text{sentiment}} \times (\text{AVG}(\text{sentiment_score}) + 1))$$

Este planteamiento teórico permite cambiar la escala total o los pesos del índice en cualquier momento simplemente modificando los parámetros de entrada del sistema, garantizando la flexibilidad de la métrica sin necesidad de alterar la estructura lógica de la consulta o el código del software.

4. Técnicas y herramientas

Este capítulo presenta las metodologías de trabajo, los entornos de desarrollo y el conjunto de herramientas utilizadas para construir el proyecto. Para cada elemento del sistema, se describen sus funciones esenciales, la justificación de su elección frente a otras alternativas del mercado y cómo se integran dentro del flujo continuo de datos del pipeline.

4.1. Metodología de trabajo

La organización y el desarrollo del proyecto se han estructurado siguiendo un enfoque ágil basado en Sprints o ciclos de trabajo de aproximadamente dos semanas de duración. Esta metodología consiste en fragmentar el objetivo general del desarrollo en pequeñas tareas más manejables que se puedan planificar, programar y probar en plazos de tiempo reducidos.

Para gestionar este flujo de trabajo de forma visual se ha empleado el sistema Kanban. Las tareas se representan en tarjetas individuales que se mueven a lo largo de un tablero dividido en columnas según su estado actual: pendientes de inicio, en proceso de desarrollo, en fase de pruebas o finalizadas correctamente. Además, para medir la carga de trabajo de cada Sprint, se han asignado puntos de esfuerzo abstractos a cada tarea. Esto permite flexibilizar el calendario de trabajo y adaptar el desarrollo al ritmo real del proyecto, priorizando siempre la estabilidad de los componentes críticos antes de avanzar.

4.2. Sistema de control de versiones (Git)

Git es un sistema de control de versiones que funciona a nivel local en el ordenador de desarrollo. Su objetivo principal en este proyecto ha sido realizar un seguimiento histórico de todos los avances y cambios realizados en el código fuente del software a lo largo del proyecto.

A diferencia de desarrollos en equipo donde se gestionan múltiples ramas paralelas, en este trabajo se ha utilizado Git de una forma lineal y directa sobre la línea principal de código. Su utilidad real ha sido registrar de manera ordenada cada modificación en los scripts de Python y en los archivos de configuración dentro del equipo local. Esto permite disponer de un historial seguro de versiones anteriores al que poder recurrir de inmediato si un cambio nuevo falla, sirviendo además como la herramienta clave para empaquetar y enviar todo el trabajo hacia el repositorio remoto de forma fácil, cómoda y rápida.



Figura 4.1: Logotipo del sistema de control de versiones Git.

4.3. Plataforma central de desarrollo y automatización (GitHub)

GitHub constituye el nodo central sobre el que se apoya toda la infraestructura, la gestión de tareas y la automatización del proyecto, unificando diferentes procesos en un único entorno:



Figura 4.2: Logotipo de la plataforma GitHub.

Gestión de tareas y repositorio

A través de las extensiones de GitHub Projects y Zenhub se implementó el tablero Kanban del proyecto, permitiendo asociar directamente las tarjetas de

tareas con las líneas de código modificadas. Asimismo, la plataforma funciona como el repositorio remoto del proyecto, asegurando el almacenamiento seguro de los scripts y el control de los cambios síncronos combinados con Git local.



Figura 4.3: Logotipo de la plataforma de gestión Zenhub.

Automatización del pipeline (GitHub Actions)

Para lograr que el sistema funcione de forma autónoma sin depender de un ordenador físico encendido, se evaluó el uso de Google Cloud Functions. Sin embargo, este servicio se descartó debido a su límite estricto de ejecución de 9 minutos por función y a que Google cobra por la memoria RAM consumida mientras el proceso está activo.

En su lugar se eligió GitHub Actions, configurado mediante un workflow que permite programar la ejecución de forma cíclica a través de un cron personalizable, realizando las ejecuciones periódicas del scraper de forma completamente automática [5]. Esta elección se justifica porque ofrece hasta 2.000 minutos mensuales de computación gratuita para repositorios públicos, permite tiempos de ejecución continuos de hasta 6 horas y regala una máquina virtual con 7 GB de memoria RAM, siendo una opción claramente mejor de automatización que Google Cloud Functions.



Figura 4.4: Logotipo del motor de automatización GitHub Actions.

Seguridad y credenciales (GitHub Secrets)

Debido a que el script automático en la nube debe conectarse de forma autónoma con Google Cloud para escribir datos en la base de datos, se utiliza la funcionalidad de GitHub Secrets. Esta herramienta cifra la contraseña en los servidores de GitHub con el nombre `GCP_SA_KEY` y solo se la entrega de forma segura al motor de Actions en el momento exacto de la ejecución del programa.

Debido a que el script automático en la nube debe ejecutarse solo y conectarse de forma autónoma tanto con el servicio de extracción como con Google Cloud, el sistema requería el uso de claves de acceso seguras utilizando la funcionalidad de GitHub Secrets.

Esta herramienta cifra las credenciales en los servidores de GitHub y solo se las entrega de forma segura al motor de Actions en el momento exacto en el que arranca el programa. En este proyecto se han configurado dos secretos esenciales:

- **Token de Apify:** La clave privada necesaria para autenticar el script de Python con la plataforma de extracción.
- **Llave de Google Cloud (`GCP_SA_KEY`):** El archivo JSON de la cuenta de servicio que da permiso al pipeline para entrar en Google Cloud y escribir de forma directa los nuevos datos dentro de la tabla de BigQuery.

4.4. Entorno de desarrollo local (Visual Studio Code)

La escritura, pruebas locales y edición de todos los componentes se han realizado sobre el entorno de desarrollo integrado Visual Studio Code (VSCode). Se eligió esta herramienta frente a otras alternativas pesadas por su ligereza en memoria y por su capacidad para unificar en una sola interfaz la edición de los archivos de código, la visualización de la estructura de las carpetas, el uso de extensiones de depuración y la terminal de comandos del sistema operativo local.



Figura 4.5: Logotipo del entorno de desarrollo Visual Studio Code.

4.5. Herramienta de extracción (Apify)

La elección del mecanismo para descargar la información de Google Maps fue uno de los debates técnicos más importantes del proyecto. Se evaluaron cuatro alternativas principales antes de tomar la decisión definitiva:

- **API oficial de Google Places:** Se descartó por completo debido a tres problemas insalvables: limita la descarga a un máximo de 5 reseñas por sitio, tiene un coste económico alto por cada petición y sus términos legales prohíben almacenar sus datos en una base de datos propia por más de 30 días.
- **Script propio con Playwright:** Planteado originalmente como el Plan A. El código utilizaba selectores basados en roles de accesibilidad para esquivar los cambios continuos que hace Google en el diseño de su interfaz. Sin embargo, se descartó por completo debido a la imposibilidad técnica de crear un capturador propio capaz de superar las estrictas medidas de seguridad de una empresa como Google, que detecta y bloquea de inmediato las extracciones masivas de datos mediante sistemas antibot y controles de dirección IP.
- **Servicios externos de pago (Outscraper, SerpApi o ValueSerp):** Estas APIs entregan la información de forma directa, pero funcionan como una "caja negra" sin control sobre el proceso y suelen generar fallos al exportar a formatos tradicionales como CSV debido a que las comas de los textos largos rompen las tablas.
- **Plataforma Apify (Opción elegida):** Se seleccionó como la herramienta de extracción central utilizando el componente desarrollado por *Compass* [4], que es el más completo y económico para cuentas gratuitas.
- **Plataforma Apify (Opción elegida):** Se seleccionó como la herramienta de extracción central utilizando los componentes desarrollados por *Compass*, el scraper de POIs [4] y el scraper de reseñas [3].

La elección de Apify se justifica porque lanza un navegador automático en un servidor propio que entra en la web, imita las acciones de una persona haciendo desplazamientos hacia abajo para forzar la carga de todas las reseñas de la pantalla y las lee directamente. Al trabajar de forma nativa en formato JSON, se garantiza la integridad de los textos sin peligro de que los caracteres especiales rompan las columnas de la base de datos. Además, ofrece una librería oficial para Python y funciona mediante un sistema cerrado de créditos que detiene el script si el saldo se agota, aportando una seguridad que evita facturas imprevistas.



Figura 4.6: Logotipo de la plataforma de extracción web Apify.

4.6. Infraestructura de almacenamiento (Google BigQuery)

Para guardar toda la información, se descartó el uso de bases de datos relacionales tradicionales locales (como MySQL en Laragon) porque obligaban a mantener un servidor físico encendido y requerían exportar archivos intermedios de forma manual para actualizar los gráficos. En su lugar, se eligió Google BigQuery.

BigQuery es un almacén de datos analítico en la nube, totalmente gestionado y optimizado para ejecutar consultas complejas sobre grandes volúmenes de información. Su elección dentro de la arquitectura del proyecto se justifica por los siguientes criterios técnicos y operativos:

- **Accesibilidad y automatización en la nube:** Al ser un servicio basado en la nube, elimina por completo la necesidad de mantener un servidor físico local encendido de forma continua. Esto permite que los scripts automáticos guarden la información directamente en internet, logrando que todo el sistema de información funcione de manera autónoma en todo momento sin depender de un ordenador personal.
- **Conexión nativa con el cuadro de mando:** Al pertenecer al mismo ecosistema de herramientas de Google que Looker Studio, la

comunicación entre la base de datos y la plataforma de visualización es directa y transparente. Esto elimina la necesidad de descargar y subir manualmente archivos intermedios en formato CSV desde el entorno local, permitiendo que el Boletín de Turismo consulte la base de datos en tiempo real y permanezca actualizado de forma automática.

- **Viabilidad económica (Modalidad Sandbox):** BigQuery ofrece una capa gratuita permanente que permite almacenar hasta 10 GB de datos y procesar hasta 1 TB de consultas mensuales sin coste. Esta capacidad es más que suficiente para gestionar el histórico de puntos de interés y reseñas analizadas en el proyecto a coste cero.



Figura 4.7: Logotipo del almacén de datos Google BigQuery.

4.7. Contenedorización (Docker)

Para asegurar la portabilidad del proyecto se ha utilizado Docker mediante la configuración de un archivo `Dockerfile` y un fichero de dependencias `requirements.txt`.

La utilización de Docker se justifica porque crea un entorno estándar aislado y reproducible. Si el entorno de Python local del ordenador de desarrollo se actualiza o rompe sus librerías, el sistema sigue funcionando correctamente dentro del contenedor independiente. El `Dockerfile` define el entorno estándar del proyecto, asegurando que el scraper funcione en cualquier sistema.



Figura 4.8: Logotipo de la plataforma de contenedores Docker.

4.8. Lenguajes de programación

La construcción, automatización y documentación del proyecto ha requerido la intervención de diferentes lenguajes:

- **Python 3.13:** El lenguaje de programación principal utilizado para escribir todos los scripts que componen la lógica del pipeline.
- **SQL (Google Standard SQL):** Es el lenguaje de consultas utilizado en BigQuery. Se ha usado constantemente para crear las tablas, insertar valores manuales de prueba, hacer consultas **SELECT** para comprobar los datos y, dentro del propio código de Python, para lanzar las consultas de subida de los nuevos datos extraídos.
- **LaTeX:** El lenguaje utilizado para escribir, estructurar y dar formato profesional a la memoria y los anexos de la documentación.
- **YAML:** El formato de datos utilizado para redactar el workflow de GitHub Actions, especificando los pasos y los horarios del servidor automático.
- **Markdown:** Lenguaje empleado para redactar los archivos (README) informativos del repositorio del proyecto.

4.9. Bibliotecas de procesamiento de datos y conectores

Para realizar las tareas de extracción, limpieza, traducción y análisis dentro del código de Python, se han utilizado las siguientes librerías:

- **apify-client:** Es la librería oficial de Apify para Python. Se encarga de inicializar el cliente operativo mediante el token, lanzar los actores de Google Maps en la nube y descargar los resultados de puntos de interés y reseñas de forma incremental en formato JSON [1].
- **google-cloud-bigquery:** Conector oficial de Google Cloud Platform. Permite que el script `bigquery_loader.py` se autentique de forma segura, cree las tablas temporales de carga y ejecute de forma directa las sentencias SQL y las fusiones mediante **MERGE** [8].

- **pysentimiento:** Es la librería encargada del análisis de sentimiento de los textos de las reseñas. El modelo calcula las probabilidades de que una reseña sea positiva, neutra o negativa, permitiendo al script realizar el cálculo matemático de restar la probabilidad negativa a la positiva para obtener una nota continua en el rango exacto de $[-1, 1]$ [14].
- **langdetect:** Biblioteca utilizada dentro del módulo de transformación para identificar automáticamente el idioma original en el que el turista escribió la reseña, sirviendo de filtro previo antes de decidir si el texto necesita traducción.
- **deep-translator:** Herramienta utilizada para conectar con el motor de Google Translate de forma gratuita. Traduce automáticamente al idioma pedido los textos de las opiniones extranjeras detectadas por **langdetect**, permitiendo almacenar en la base de datos tanto la versión original como la traducida [2].
- **gender-guesser:** Librería de detección de género. Analiza el primer nombre del autor de la reseña y lo clasifica automáticamente como "Masculino." "Femenino". En caso de nombres ambiguos, perfiles de empresas o cuentas privadas, permite inyectar el valor "Desconocido" por defecto configurado en el sistema.

4.10. Herramienta de cuadro de mando y visualización (Looker Studio)

La fase de explotación final de los datos se ha desarrollado sobre la plataforma Looker Studio. Aunque se estudió el uso de herramientas líderes del mercado como Power BI, su elección final vino impuesta por una restricción técnica del servidor del propio Observatorio de Turismo, que exigía para su servidor la utilización de Looker Studio en su lugar.

De este modo, Looker Studio se justifica como la única solución viable por los siguientes motivos:

- **Gratuidad y acceso web:** Es una herramienta totalmente gratuita basada en la nube que permite diseñar gráficos interactivos sin costes de licencias.
- **Conexión nativa con BigQuery:** Al pertenecer ambas herramientas al ecosistema de Google, la sincronización de las tablas de datos es

directa y automática, lo que permite que los gráficos se actualicen solos en cuanto el pipeline añade nuevos registros, sin necesidad de abrir el portátil ni ejecutar nada.

- **Facilidad de integración:** Permite generar códigos de incrustación seguros para insertar el cuadro de mando de manera transparente dentro del portal web de la institución.



Figura 4.9: Logotipo de la herramienta de visualización Looker Studio.

4.11. Herramientas de diseño gráfico y diagramas (Draw.io)

Para la fase de diseño de la arquitectura del sistema y la creación de los diagramas técnicos de la memoria, se ha utilizado la herramienta web Draw.io. Esta aplicación permite diseñar de forma visual y personalizada la estructura de los componentes informáticos mediante plantillas estándar.



Figura 4.10: Logotipo de la herramienta de diagramas Draw.io.

4.12. Herramientas de asistencia (Gemini)

Se ha empleado la herramienta de inteligencia artificial Gemini como recurso de apoyo consultivo durante el desarrollo del proyecto. Su uso ha consistido en resolver dudas técnicas de programación y a agilizar el diseño del pipeline.

Entre los casos concretos de uso se incluye la corrección de errores de la terminal al ejecutar el sistema, el soporte para estructurar algunas consultas SQL dentro de BigQuery, la resolución de dudas teóricas sobre el proyecto y la ayuda con el formato de las etiquetas de $\text{\LaTeX}$ para la maquetación de esta memoria.



Figura 4.11: Logotipo del asistente técnico Gemini.

5. Aspectos relevantes del desarrollo del proyecto

En este capítulo se explican los puntos más importantes del desarrollo práctico de este trabajo, los problemas reales de ingeniería de software que surgieron durante el diseño del pipeline y cómo se resolvieron para conseguir una herramienta segura, automática y optimizada en costes.

5.1. Estructura y división del código en archivos independientes

En primer lugar, se organizó el código en un único script que contenía todas las funciones del proyecto. A medida que fue creciendo, el programa empezaba a ser imposible de mantener, de entender y de reparar si algo fallaba. Para evitar este problema, se tomó la decisión de adoptar el diseño de un algoritmo ETL dividiéndolo de forma limpia en cuatro archivos de Python independientes. Cada uno de ellos se encarga de una fase concreta del proceso con sus respectivas funciones:

- **main_scraper_pipeline.py:** Es el motor que arranca todo el programa. Su tarea es leer el archivo de configuración externa, poner en marcha el sistema de registros de errores (logs) y controlar la sucesión de bucles que van recorriendo la provincia de Burgos. Se encarga de llamar a los demás archivos en el orden correcto y de controlar que el flujo no se detenga.
- **apify_extractor.py (Módulo de extracción):** Este archivo concentra todo el código que se conecta con la API de Apify. Su función

es pedirle a la plataforma que inicie el proceso de copia de datos de Google Maps, vigilar cuándo termina de trabajar y descargarse la información de los puntos de interés o de las reseñas en bruto. Al dejar esta lógica aquí, si el día de mañana Apify cambia su forma de conectarse, solo habrá que parchear este archivo sin tocar el resto del sistema.

- **review_transformer.py (Módulo de transformación):** Es el encargado de procesar los datos en bruto de las reseñas que se acaban de descargar. Utiliza librerías para limpiar los textos de las opiniones, traducir de forma automática las reseñas que dejen los turistas extranjeros, averiguar el género del usuario analizando su nombre de pila y aplicar el modelo de análisis de sentimiento.
- **bigquery_loader.py (Módulo de carga y consultas):** Es el script que se conecta directamente con el almacén de datos en la nube de Google BigQuery. Contiene las consultas SQL necesarias tanto para insertar los nuevos datos como para preguntar a la base de datos en qué estado se encuentran las tablas antes de empezar a trabajar.

Esta división permite que el código sea modular. Si por ejemplo se quiere mejorar el filtro de limpieza de los textos, solo hay que modificar el archivo `transformer.py`, garantizando que el sistema de extracción o la base de datos sigan funcionando exactamente igual y sin riesgo de romper nada.

5.2. Protección contra fallos y sistema de checkpoints)

Al ser un sistema que utiliza en gran medida herramientas que trabajan en la nube, existe la posibilidad de que falle el internet durante la ejecución, que las APIs de terceros sufran caídas, que los servidores externos se saturen y que las conexiones sufran microcortes. Si el programa se corta por culpa de un fallo de red, sería un desastre económico y de tiempo tener que empezar desde el principio en la siguiente ejecución, ya que se volverían a gastar créditos de la API de Apify de forma totalmente innecesaria.

Funcionamiento del guardado por lotes

El pipeline no trabaja con toda la provincia de golpe en la memoria del ordenador, sino que el proceso se realiza de forma independiente bloque a

bloque. El script divide el trabajo en dos fases principales (Fase A para puntos de interés y Fase B para reseñas) y va guardando la información en BigQuery en cuanto termina cada lote pequeño de datos:

- **En la Fase A (POIs):** El programa recorre los municipios y, dentro de cada uno, va categoría por categoría. En cuanto termina de extraer y procesar los puntos de interés de una categoría concreta en ese municipio, el módulo `bigquery_loader.py` guarda ese lote inmediatamente en BigQuery. Justo después, escribe un registro de control en la tabla `scraper_control` indicando el municipio, la categoría y la fecha y hora exacta del momento en el que se completa. Además, cuando se terminan todas las categorías de ese municipio, se actualiza la fecha global `last_poi_update` de extracción de POIs en la tabla de municipios.
- **En la Fase B (Reseñas):** El proceso se repite de forma individual punto de interés por punto de interés. Al terminar de procesar y aplicar el análisis de sentimiento a las opiniones de un sitio concreto, las guarda en la base de datos, actualiza su índice TORI y actualiza el campo `last_review_extraction` de ese POI con la fecha actual. Al acabar todos los sitios del municipio, actualiza también la fecha de extracción de reseñas de la tabla global de municipios.

Lógica de reinicio inteligente

Gracias a que los datos y las fechas de control se van guardando de forma inmediata en cada paso del bucle, el pipeline está completamente protegido si el programa se queda colgado a mitad de la noche por un corte de luz o de red.

Al volver a arrancar el script, lo primero que hace el archivo `main_scraper_pipeline.py` es pedirle a `bigquery_loader.py` que lea los tiempos de control en BigQuery. El programa utiliza una serie de fechas y márgenes de días configurables desde el archivo de configuración para comprobar qué elementos ya se han procesado recientemente. Con esta información, el script evita que los bucles vuelvan a pasar por todos aquellos elementos que ya han sido completados dentro del margen establecido. De este modo, se impide repetir el trabajo de forma innecesaria, ahorrando horas de procesamiento y protegiendo el dinero invertido en los créditos de las APIs.

Además, al seguir estrictamente el mismo orden de extracción (marcado por la ordenación de las consultas SQL a la base de datos), el pipeline

continuará el trabajo exactamente en el punto exacto en el que se quedó, haciendo que la ejecución sea completamente previsible, ordenada y fácil de controlar para el administrador del sistema.

5.3. Desacoplamiento y configuración externa del sistema

Como buena práctica de ingeniería de software, se ha aislado por completo el comportamiento del programa de la lógica de programación mediante el uso del archivo externo `config.json`. Este fichero permite que cualquier usuario o técnico del Observatorio personalice y configure todos los aspectos de la ejecución desde fuera, sin tener que modificar ni una sola línea de código en los scripts de Python. A través de esta estructura centralizada se gestionan parámetros críticos de todo el sistema: desde las credenciales y rutas de las tablas de Google BigQuery, hasta las reglas de negocio del proyecto, como los umbrales de población para la búsqueda adaptativa, los filtros para forzar la ejecución en un único municipio o categoría concretos, y la configuración técnica del comportamiento del extractor, el traductor y el nivel de detalle de los archivos de *logging*.

5.4. Optimización de costes de la API según los habitantes del municipio

Uno de los mayores retos técnicos y de gestión de este proyecto ha sido el control del coste económico de la API de Apify. El scraping masivo en Google Maps tiene un coste elevado por cada elemento extraído, y lanzar búsquedas idénticas en todos los rincones de la provincia por igual sería una gestión nefasta de los recursos. En la provincia de Burgos hay municipios muy grandes con miles de puntos de interés (como Burgos capital) y pueblos muy pequeños con apenas unas pocas decenas de habitantes donde la oferta turística y comercial es muy reducida.

Si se programa el bucle para que busque siempre todas las categorías registradas independientemente del municipio, la API de Apify gastaría tiempo y créditos en hacer barridos geográficos que no devuelven ningún resultado.

Para solucionar este problema de eficiencia, se diseñó una regla de optimización adaptativa basada en la población que cruza el tamaño de la localidad con un nivel de búsqueda asignado a cada categoría (del 1 al 3).

Al leer la lista de municipios, el script comprueba el número de habitantes y adapta la intensidad de la búsqueda de forma dinámica en tres niveles (rural, semi-urbano y urbano). En municipios urbanos (de nivel 3), el flujo de ejecución activa el listado completo y exhaustivo de categorías, buscando todo tipo de etiquetas específicas para capturar toda la información. En los municipios semi-urbanos (de nivel 2), el sistema descarta las categorías más específicas e innecesarias que puedan cubrir otras más generales pero manteniendo un abanico amplio de opciones, mientras que en los municipios rurales (de nivel 1), el programa altera su comportamiento y recorta la lista de búsqueda dejando únicamente las categorías más básicas y esenciales (las de nivel 1).

Al dejar fuera las búsquedas comerciales complejas en zonas rurales donde no existen, se optimiza al máximo la cuota de la API y se reduce drásticamente el coste final de operación.

5.5. Descubrimiento y guardado dinámico de nuevas categorías

El objetivo principal de esta funcionalidad es ir recopilando poco a poco todas las categorías de puntos de interés reales que existen en la provincia de Burgos para que, con cada nueva ejecución del pipeline, los resultados sean mejores, más precisos y el sistema sea capaz de encontrar nuevos puntos de interés más específicos.

Para lograrlo, el script incluye una lógica que va ampliando la base de datos de forma automática: cuando se encuentra un POI con una categoría de Google Maps que no estaba registrada, el sistema le asigna un identificador único y, de manera totalmente automática, el programa la guarda dinámicamente dentro de la taxonomía de la categoría con la que se estaba realizando el rastreo en ese momento, además de establecerle su mismo nivel de búsqueda, ya que se presupone una similitud o afinidad entre ambas.

Esto permite que el pipeline sea cada vez más inteligente y completo de forma autónoma, garantizando además que el cuadro de mando de Looker Studio actualice sus filtros visuales por sí solo y sin necesidad de tocar una sola línea de código de la aplicación.

5.6. Logging

Al dejar el pipeline ejecutándose de forma automática, los mensajes de la consola se acaban perdiendo o son imposibles de encontrar. Por este motivo, se eliminaron todas estas impresiones básicas del código (`print()`) y se implementó un sistema de registros estructurado mediante el módulo nativo de *logging*. La gran ventaja es que ahora, cada vez que ocurre un evento, se guarda automáticamente la fecha, la hora exacta con milisegundos y la línea del script donde ha saltado [12].

Además, el sistema permite filtrar las trazas según las necesidades del momento: se utiliza el nivel *DEBUG* para ver flujos internos detallados durante el desarrollo, el nivel *INFO* para el seguimiento normal del pipeline, el nivel *WARNING* para capturar advertencias de rendimiento o anomalías que no sean críticas, y el nivel *ERROR* para registrar únicamente las excepciones graves que llegan a colgar o interrumpir algún módulo del programa.

Para que esta monitorización sea realmente útil en el día a día, se configuró el *logger* para que los registros se guarden a la vez en la consola y en archivos físicos ordenados por fecha dentro de la carpeta `logs`. Al hacerlo así, la información no se borra al apagar o reiniciar el contenedor, sino que se crea un historial permanente. Esto facilita mucho el mantenimiento del sistema, ya que permite revisar de forma rápida qué ha pasado o dónde ha fallado el programa en cualquier ejecución anterior sin tener que estar delante de la pantalla mientras se ejecuta.

Capítulo 6

Trabajos Relacionados

En este capítulo se analizan los antecedentes y los proyectos relacionados con este trabajo. En primer lugar, se compara este proyecto con el TFG realizado el año pasado en la Universidad de Burgos, que abordó el mismo problema pero con un enfoque tecnológico diferente. En segundo lugar, se explica cómo se alinea este sistema con los planes de turismo digital que existen actualmente en España, concretamente con el programa de Destinos Turísticos Inteligentes (DTI) gestionado por el gobierno [16].

6.1. Comparativa con el proyecto anterior

El antecedente más directo de este trabajo es el TFG titulado "*Digitalización del Boletín de Turismo del Observatorio de la Provincia de Burgos*", presentado en julio de 2025 [11]. Ese trabajo dio un primer paso para recoger opiniones de usuarios de internet sobre los recursos turísticos de la provincia. Sin embargo, la forma de resolver el problema a nivel técnico fue muy distinta. Para entender por qué la solución actual mejora a la anterior, se detallan las diferencias clave en tres aspectos:

Almacenamiento y costes

El proyecto del año pasado utilizaba *Azure SQL Database* (el motor de base de datos relacional de Microsoft) en la nube, que se gestionaba visualmente con SQL Server Management Studio (SSMS). Aunque cumplía con su función de guardar los datos, este tipo de bases de datos genera costes fijos mensuales en Azure por mantener el servidor aprovisionado y encendido continuamente.

Para este proyecto se ha cambiado la estrategia utilizando *Google BigQuery*. Al ser una herramienta orientada al análisis de grandes volúmenes de datos y de tipo serverless (sin servidor fijo), solo se paga por el almacenamiento real y por las consultas que se ejecutan, eliminando los costes por inactividad. Además, para que la actualización de los datos sea más rápida y eficiente, se han configurado vistas SQL y sentencias *MERGE* que solo añaden los datos nuevos sin tener que procesar todo desde cero.

Automatización del flujo de trabajo (ETL)

El flujo de trabajo del proyecto anterior estaba dividido en varias partes y no era automático. Tenía tres scripts de Python independientes para extraer y limpiar los datos. Después, para tareas más avanzadas, se dependía de ejecutar manualmente varios notebooks en la plataforma *Google Colab*. Por último, esos datos se subían de forma independiente a la base de datos.

En este nuevo proyecto se ha unificado todo el proceso. Ahora no hay que ejecutar varios scripts ni usar herramientas externas a mano; un único pipeline programado en Python se encarga de todo de forma secuencial. La extracción de datos con Apify y la transformación y enriquecimiento de las reseñas se ejecutan de principio a fin, guardando los resultados directamente en la nube sin necesidad de que un administrador supervise cada paso.

Visualización e integración en el observatorio

El TFG anterior utilizaba *Power BI* para crear el cuadro de mando y se subía a una web creada con *Power Pages*. Esto añade costes de licencias comerciales y complica la integración con la web real del Observatorio de Turismo de Burgos. El servidor del observatorio tiene restricciones de administración y no permite ejecutar scripts de Python en el backend.

Para solucionar esto, en este proyecto se ha utilizado *Looker Studio*, que es una herramienta gratuita y se conecta de forma directa con BigQuery. Como Looker Studio permite insertar los gráficos fácilmente en cualquier web mediante un código estándar, el cuadro de mando se puede añadir directamente en el servidor del observatorio.

6.2. Marcos de referencia nacionales

Este proyecto se enmarca dentro de la estrategia actual del Gobierno de España para digitalizar el sector turístico. El principal referente en este

ámbito es el **Programa de Destinos Turísticos Inteligentes (DTI)**, impulsado por la Secretaría de Estado de Turismo y gestionado por SEGIT-TUR. Este programa evalúa a los destinos en cinco áreas básicas: gobernanza, innovación, tecnología, accesibilidad y sostenibilidad.

La herramienta desarrollada en este TFG se alinea con los apartados de tecnología e innovación por los siguientes motivos:

- **Sistemas de Inteligencia Turística (SIT):** El modelo DTI impulsa la creación de herramientas que recojan de forma automática datos e indicadores de interés para ayudar a las instituciones a tomar decisiones estratégicas. Este pipeline cumple esa función a nivel provincial al automatizar por completo la captura de datos de Google Maps, una de las plataformas que más volumen de opiniones y recursos turísticos concentra en la actualidad.
- **Medición de la reputación online:** Las estadísticas oficiales (como las del INE) solo miden datos cuantitativos, como el número de personas que se alojan en un hotel. El programa DTI pide que también se mida la calidad de la experiencia del turista. Al extraer reseñas de Google Maps y calcular el Índice de Reputación Online Turística (TORI), este proyecto permite conocer de forma objetiva qué opinan los turistas sobre los monumentos y la hostelería de Burgos, entre otros puntos de interés.

7. Conclusiones y Líneas de trabajo futuras

En este último capítulo se exponen las conclusiones derivadas del diseño, desarrollo e implementación del proyecto. Asimismo, se incluye una valoración crítica del sistema actual y se proponen las principales líneas de trabajo futuras para continuar expandiendo y evolucionando la herramienta.

7.1. Conclusiones del proyecto

El desarrollo de este Trabajo de Fin de Grado ha permitido cumplir con éxito el objetivo principal: automatizar la recopilación y el procesamiento de las opiniones turísticas de la provincia de Burgos para su digitalización en el Boletín de Turismo.

Desde el punto de vista del Observatorio de Turismo de Burgos, se ha logrado aportar una solución práctica y directa que permite ver la realidad del sector desde las opiniones de los usuarios, lo que ayuda a que la gestión de la institución deje de depender de la recopilación manual de datos tradicionales, pasando a basarse en un flujo constante de información real y objetiva.

A nivel personal, este proyecto ha supuesto un enorme reto de aprendizaje que me ha permitido enfrentarme por primera vez a un entorno de desarrollo real y aplicar los conocimientos de la carrera de una forma totalmente práctica. Trabajar e integrar tecnologías tan diversas como Python, las bases de datos en la nube con Google BigQuery, los paneles visuales de Looker Studio y el uso de APIs externas de extracción me ha ayudado a comprender cómo conectar diferentes herramientas para construir un sistema completo y robusto.

Por otro lado, la principal limitación que ha condicionado el desarrollo práctico del proyecto ha sido el coste económico de la API de Apify. Al implicar un gasto por cada elemento extraído de Google Maps, no fue posible realizar tantas ejecuciones de prueba como me hubiera gustado, ni tampoco descargar todo el histórico completo de la provincia de una sola vez como prueba final. Sin embargo, tener que lidiar con esta restricción presupuestaria real obligó a buscar soluciones de diseño más eficientes, lo que dio pie al desarrollo del sistema de checkpoints y a la optimización según la población de los municipios. Al final, enfrentarme a estos problemas de costes me ha aportado un gran aprendizaje como ingeniero, ya que me ha enseñado que el software no solo debe funcionar, sino que debe ser eficiente y adaptarse a los recursos disponibles en el mundo real.

7.2. Líneas de trabajo futuras

A partir de la arquitectura consolidada en este proyecto, se abren varias vías de desarrollo que permitirán expandir las capacidades de la plataforma y aportar nuevas capas de valor añadido al Observatorio de Turismo:

Ampliación de la cobertura geográfica y escalabilidad regional

Una de las continuaciones más naturales del proyecto es romper la barrera provincial y escalar el pipeline para dar cobertura al resto de la comunidad autónoma de Castilla y León. Dado que la taxonomía de categorías, la estructura de almacenamiento en BigQuery y la lógica de extracción en Python se han diseñado siguiendo un patrón completamente genérico, el sistema está preparado para incorporar nuevas provincias sin alterar su núcleo. Bastaría con alimentar la base de datos con los listados municipales correspondientes para que el pipeline comenzara a recopilar y enriquecer el boletín de turismo a nivel regional de forma inmediata.

Desarrollo de una interfaz de usuario unificada

Actualmente, el sistema funciona de manera desacoplada: el pipeline se configura y ejecuta mediante scripts de código y archivos de configuración, mientras que la visualización de los datos se realiza de manera externa en Looker Studio. Una línea de trabajo futura de gran interés es el desarrollo de una aplicación web o panel de administración unificado. Esta interfaz permitiría a los técnicos de la institución gestionar el archivo de configuración

visualmente, lanzar o programar las ejecuciones del pipeline con un solo clic y visualizar los cuadros de mando analíticos directamente en la misma plataforma, centralizando toda la operativa en un único entorno de trabajo más moderno y accesible.

Evolución del motor de ejecución y modos de operación avanzados

Para dotar al pipeline de una mayor versatilidad y capacidad de adaptación ante diferentes escenarios de uso, se plantea la evolución del motor de ejecución mediante la inclusión de modos de ejecución avanzados seleccionables desde la configuración externa. Esta ampliación se estructuraría en dos grandes bloques funcionales:

En primer lugar, un **selector de objetivos por fases**, que permita al usuario decidir mediante variables si desea ejecutar el flujo completo del pipeline, realizar únicamente tareas de descubrimiento de nuevos puntos de interés (Fase A) o centrarse exclusivamente en la extracción de nuevas reseñas para los establecimientos que ya tenemos registrados en la base de datos (Fase B). Esto daría un control absoluto sobre el uso de los recursos según las necesidades analíticas de cada momento.

En segundo lugar, un **optimizador de velocidad para entornos rurales**. Para agilizar las ejecuciones en municipios pequeños de nivel 1, se propone el desarrollo de un modo de ejecución rápido alternativo al actual. Mientras que el modo normal itera de forma consecutiva por cada una de las categorías básicas, el modo rápido permitiría lanzar una única consulta geográfica global unificada en la API para capturar todos los recursos del pueblo de una sola vez ahorrando muchas horas de ejecución, pero obteniendo resultados menos precisos. El pipeline controlaría este comportamiento mediante una variable marcada en la configuración, permitiendo al administrador elegir entre un rastreo lento y exhaustivo por categorías o una actualización exprés en núcleos de baja densidad poblacional.

Bibliografía

- [1] Apify Technologies s.r.o. apify-client: Official apify api client library for python. <https://pypi.org/project/apify-client/>, 2026. [Internet; registro de metadatos en PyPI; consulta: 2-julio-2026].
- [2] Nidhal Baccouri. deep-translator: A flexible tool to translate between different languages using multiple translators. <https://pypi.org/project/deep-translator/>, 2026. [Internet; registro de metadatos en PyPI; consulta: 2-julio-2026].
- [3] Compass via Apify. Google maps reviews scraper: Bulk extract detailed reviews, ratings, and reviewer metadata. <https://apify.com/compass/google-maps-reviews-scraper>, 2026. [Internet; registro de la herramienta en la tienda de actores de Apify; consulta: 2-julio-2026].
- [4] Compass via Apify. Google maps scraper: Extract data from google maps places, reviews, and images. <https://apify.com/compass/google-maps-scraper>, 2026. [Internet; registro de la herramienta en la tienda de actores de Apify; consulta: 2-julio-2026].
- [5] GitHub Inc. Github actions documentation: Automating workflows and ci/cd pipelines. <https://docs.github.com/en/actions>, 2026. [Internet; documentación técnica oficial del motor de automatización; consulta: 2-julio-2026].
- [6] Google Cloud. Bigquery storage overview: Column-oriented storage architecture. https://cloud.google.com/bigquery/docs/storage_overview, 2026. [Internet; documentación de optimización analítica de almacenamiento; 30-junio-2026].

- [7] Google Cloud. What is etl (extract, transform, load)? cloud concepts. <https://cloud.google.com/learn/what-is-etl>, 2026. [Internet; sección de fundamentos y arquitectura de datos; 30-junio-2026].
- [8] Google LLC. google-cloud-bigquery: Google bigquery api client library. <https://pypi.org/project/google-cloud-bigquery/>, 2026. [Internet; registro de metadatos en PyPI; consulta: 2-julio-2026].
- [9] Hugging Face Inc. The hugging face course: Introduction to natural language processing and transformer architectures. <https://huggingface.co/course/>, 2026. [Internet; plataforma educativa oficial de NLP; 30-junio-2026].
- [10] MDN Web Docs. What is an api? - learn web development and http data structures. https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Client-side_web_APIs/Introduction, 2026. [Internet; guía técnica de Mozilla para desarrolladores; 30-junio-2026].
- [11] A. Núñez. Digitalización del boletín de turismo del observatorio de la provincia de burgos. <https://riubu.ubu.es/>, 2025. [Internet; Repositorio Institucional de la Universidad de Burgos (RIUBU); 5-julio-2026].
- [12] Python Software Foundation. Guía del usuario de logging (logging howto) — documentación de python. <https://docs.python.org/es/3/howto/logging.html>, 2026. [Internet; guía práctica oficial de configuración de registros; consulta: 4-julio-2026].
- [13] Juan Manuel Pérez, Juan Carlos Giudici, and Franco M. Luque. pysentimiento: A python toolkit for sentiment analysis and social media text mining. <https://github.com/pysentimiento/pysentimiento>, 2026. [Internet; repositorio oficial de código fuente y documentación en GitHub; 30-junio-2026].
- [14] Juan Manuel Pérez, Juan Carlos Giudici, and Franco M. Luque. pysentimiento: A python toolkit for sentiment analysis and social media text mining. <https://pypi.org/project/pysentimiento/>, 2026. [Internet; registro de metadatos en PyPI; consulta: 2-julio-2026].
- [15] Real Python Team. A practical introduction to web scraping in python. <https://realpython.com/python-web-scraping-practical-introduction/>, 2025. [Internet; artículo técnico de programación; consulta: 30-junio-2026].

- [16] SEGITTUR. Metodología de destinos turísticos inteligentes (dti) - innovación y tecnología. <https://www.segittur.es/destinos-turisticos-inteligentes/>, 2026. [Internet; portal institucional del Ministerio de Industria y Turismo; consulta: 5-julio-2026].